

Step 1: Pick your agent (pro)

Claude code - More interactive, Faster feedback

Codex - Better at taking a task working all through to completion, very good at the planning, It's able to work autonomously on a task for a longer period of time.

Just start small, an evening or two spent on it will be enough to get you started.

AI skills that can be actually transferable

You have to pay for a base subscription in order to use the coding agent (around 20 dls a month), but i think the value that you get from it will be really, really worth it.

I would avoid some of the free ones that are available out there just because they're just not quite up to par with what's available.

You really wanna be working with tools that the industry professionals use.

Start learning some of the basics with working with an agent, **don't try to learn everything at once.**

A lot of it will just come as you work with it more.

Agent fundamentals: context, context management, memory, model selection, plan mode (an evening, plus a tutorial or two) install the coding harness, the coding agent, that's everything you need to get started.

Wait list

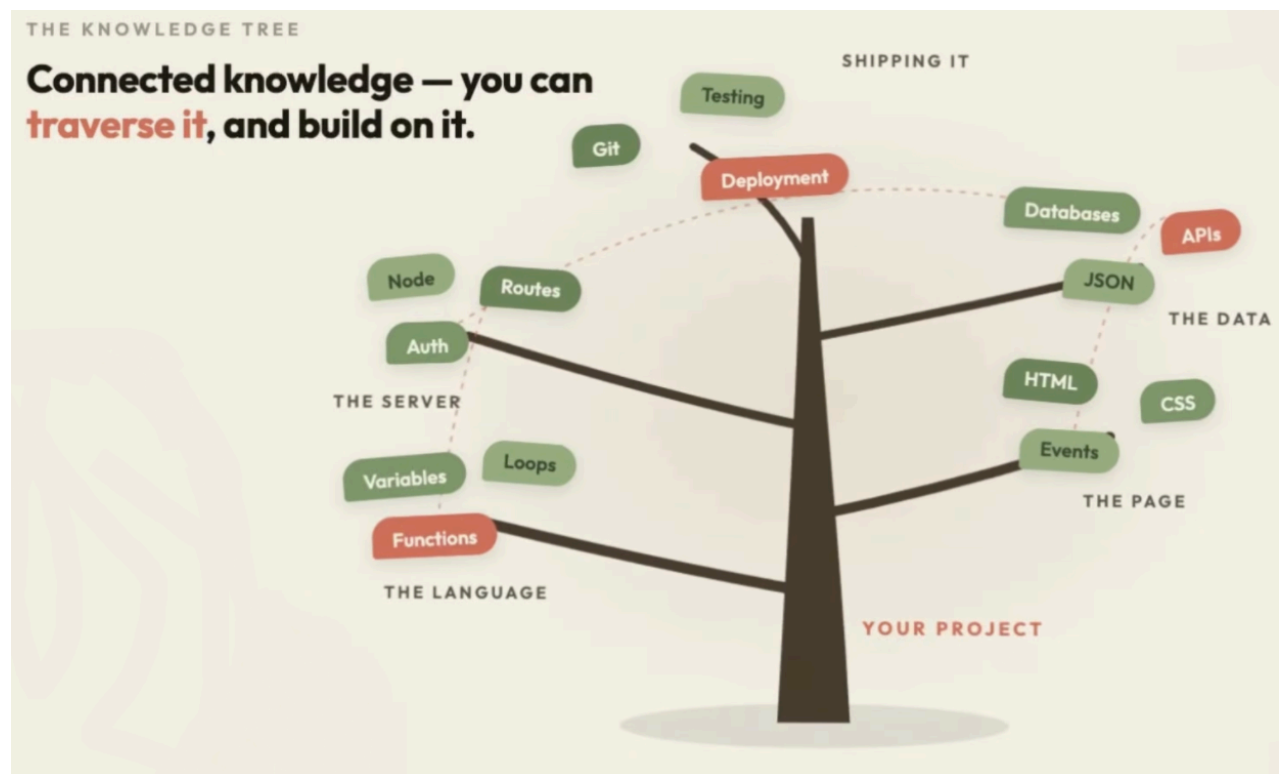
<https://learnaltitude.com/>

AI coding harness - it provides an interface that keeps track of all of that for you, visualizing your knowledge graph in a web interface plug into your coding agent. Keeping of your sessions and then give you a nice UI to interact with.

Step tracker - so you'll always know where you are in your project.

Assessment - figure out where you are in your coding experience, this will sort of seed some of the initial knowledge graphs. If you already comfortable with some of the ideas and concepts it won't try to requiz you on those things.

<https://jasonk.us/workshop>



Step 2: Pick your project

The Knowledge Tree - "why projects beat tutorials"

In this analogy every language, technology or concept you learn is a **leaf**.

Having an structure or an anchor to attach the leaves on makes it a lot easier to kind of form relationships and make the learning a lot more durable.

The trunk - a project you actually care about.

The project that you picked acts as the tree, this is the anchor that we're gonna be attaching all of these leaves onto.

You're going to start defining some of the branches (you can use AI to help you with this) walking through it step by step.

Once you have the branches then the individuals leaves that you learn become a lot easier because now they there's relationships between them, and you know where each life or which piece of the puzzle like fits where.

How to pick your project? Try to pick something that matters to you, try to solve a problem that you have. Picking a project like this serves a couple of purposes. If you pick a project that's its actually meaningful to your life, there's more motivation to continue coming back to it.

One of the best feelings is being able to see a project come to life.

If you have trouble coming up with an idea, you can actually just have ai interview you about your life. You can ask to ask you about your hobbies, your routines, your work, and have it pitch you ideas of what you can build.

Therefore you don't want to build something way too complicated (two-sided marketplace with payments, shopbuilders, like that might be too hard for a first project.

When you have an AI interview about a project you can also tell it about your general experiences with building products in software and it will pitch you ideas that are sort of appropriate for your current level.

Step 3: MVP (*Minimum Viable Product*)

the smallest thing someone can actually use

Why is picking an MVP important? One of the most valuable feedback loops for an engineer is to be able to go from an idea and all the way to the finished product.

One of the things that can get people stuck in the beginning is you end up working endlessly on a feature and it just stays on your computer forever.

Defining an MVP allows you to have your idea, very minimal set of features that you can ship to have something working end to end. And then once you have that, you can start layering out more and more features.

Identify the bare essentials, choosing the minimum set of features that you can build an app and that's gonna be your MVP.

The features didn't die - they got in line.

You might have all of these features that you want build and you can put those into the backlog and once that once you've built your first shippable product anytime you layer in one of these features you can actually see it live on your phone. Also this is just a good habit to build if you're trying to do any sort of software or startup. This is the same strategy that most startups follow is you build the very bare minimum viable product and you start showing that to users getting feedback and iterating before you start working on a ton of features that might not be used.

Step 4: Build the trunk

Starting to lay the foundations of like what are the core pieces you are working with.

This is to help your mind get a scaffolding when you start building out the branches and the leaves and this will help you anchor your knowledge.

Because the skill of software engineering has sort of moved up a level, it's important for beginners to start thinking about the system as a whole from day one

Core components of the app/project, source control, frontend, backend, database, api, where it runs (local vs deployed), deployment (opening for business).

Step 5: Build the branches

The branches are the highlevel sections or the features the horizontal slicing of how you're going to ship your project.

Your branches would be for example:

- 1.- Get it running & first page
- 2.- Make it look good & respond to clicks
- 3.- Give it some memory (database)

Each of these sections/branches start to build on top of each other

The branches are the individual features that build on top of each other.

Step 6: Define the leaves

This are the individual tasks, you start with a high level spec of where you're trying to build and then you start breaking down the individual features and then you go into task.

Start with the very very essential

Leaves its how you break down the branches

There will be learning concepts that are weaved within each of these individual tasks.

And that's where we start getting into the weeds of it.

Step 6 is where things start to get really interesting because this is where we start actually leveraging AI to help us learn.

Knowledge graph - Graph of everything that you've learned and everything that you're going to learn. And it goes all the way down to very lowlevel concepts like CSS styling, variables. And it can include higher level concepts and ideas such as git and commits

In step 6 when the AI breaks down your tasks as you start working through the tasks it will also construct a knowledge graph and it will take a look at the task and figure out what are some of the core concepts that we're going to be covering within this task and for each of those it will first check if there's already an entry for that and if not it will create an entry for it.

The power of the knowledge graph is keeping track of your status, the las time it was reviewed and then a list of evidence.

As you're mastering a concept, (example) it'll go from introduce to practicing, the next step might be something like understood and then finally mastered. Depending on the status of a certain concept, the next time that it shows up in your plan or your task, the agent can know exactly how in-depth to go into each one.

(example) if we've already introduced what variables are in task two and you have a pretty good grasp of it when it comes up again in a different task down here, AI doesn't need to teach you about it or quiz you about it as much.

Step 7: One task at a time

this is the step that you're going to be spending the majority of your time

Instructs the agent to do is to follow that checklist that we made. Constructs the knowledge graph and then provide quizzes as you're going through each of the tasks.

This solve the problem that most beginners make, which is building things way too fast with AI and not understanding any of it.

When you let the AI just solve the problem you're not learning anything. And that's a way for your code bases snowball out of control where you don't understand anything anymore.

Forcing you to slow down and look at every single line of code, quizzing yourself on every single line of code, even if that slow you down a lot, this is where the learning happens. It happens in the friction, it happens in the recall and the quizzing yourself.

Step 7 i where we introduce that friction and using this prompt going through one task at a time and making sure that we understand every single line, every single concept that's introduced within that task.

What the AI will do is introduce what that task is about, it will generate some code for you and then it will quiz you on it. Before it runs the code it will ask you "before we run this in your own words what do you expect to happen when we run through this code. This is what makes the learning truly interactive and puts you in the seat of being fully involved in the learning and as you complete each of these quizzes and you go through each of these tasks the AI goes back in and updates your knowledge graph.

in the knowledge graph the status of the concepts will been updated from introduced, practicing, understood, mastered through the completion of the tasks.

One of the biggest complaints from beginners is that they get lost and confused about know what to learn, how to learn it, and just trying to navigate everything that you have to learn.

Once you have a knowledge graph how tracks everything that you have learned so far and everything that you're going to learn, you have this simple step by step program that you follow.

Repeating step 7 through all of your sections will basically take you from a complete beginner all the way to shipping and deploying your app. Then at the end of the project, you will actually have confidence that you've not only understand every single line of code, but you've had some repetition and practice with each of the concepts.

How long does this take?

Steps 2-6: around one evening if you spend a few hours on it

Step 7: depending on how complex the project is

if you spend at least 10 hrs/week it would take a minimum of a month up to 3 months.

But if you really take the time to focus and slow down and work through it, by the end of it, you will have a really solid foundation going into whatever you do next. Making it into a portfolio piece and be able to explain every single line of code in it and you will have an understanding of the system and the whole architecture and again just a really really solid foundation as a software engineer.

As you learning this way you will also get experience of what it's like to work with a coding agent. Knowing what are some of the capabilities it has, some of the limitations, naturally learning things like context management, picking the right models for things, because on the \$20 dollar plan you'll have to try to manage your token budget.